

2026 학년도
『AI(클로드 코드) 기반 VIBE CODING 실전활용 경진대회』
프로젝트 계획서

	학과	학번	학년	이름	코딩가능여부	담당
팀장	컴퓨터공학전공	60245208	3	김세원	가능	기획 / 디자인
팀원	컴퓨터공학전공	60245214	3	송하성	가능	프론트엔드 / 보안
팀원	컴퓨터공학전공	60245212	3	민진홍	가능	백엔드 / 인프라
팀원	컴퓨터공학전공	60245209	3	김신우	가능	백엔드 / 보안

주제	VibeGuard: AI 생성 코드의 보안 취약점을 탐지하고, 테스트 기반으로 안전한 패치까지 제안하는 Multi-Agent 시스템
제안배경	AI 가 코드를 만들었다면, AI 가 만든 코드의 보안까지 검증은 누가 할 것인가? Vibe Coding 환경에서는 개발 속도가 비약적으로 빨라진 만큼, 개발자가 AI 가 생성한 코드와 의존성의 보안 취약점을 충분히 검토하지 못한 상태로 배포할 위험이 커지고 있습니다. 이번 경진대회 교육과정에서 Claude Code 의 강력한 자동화 성능을 경험하는 동시에 AI 가 짜준 코드를 어디까지 신뢰할 수 있는가?라는 의문이 생겼습니다. 이에 교육에서 학습한 Claude Code, MCP, Multi-Agent, TDD, Code Review 기술을 역으로 활용해 AI 생성 코드의 보안을 AI 가 스스로 검증하고 치유하는 시스템을 기획했습니다. [기존의 문제점] 알려진 취약점의 검증 부재: 취약 버전의 라이브러리 사용이나 입력값 검증 누락은 기능상 정상 동작할 수 있어 일반적인 단위 테스트만으로 발견하기 어렵습니다. 기존 자동화 도구의 한계: Dependabot, Snyk 등은 경고나 버전 업데이트 알림에 그치는 경우가 많으며, 버전 상향에 따른 하위 호환성 파괴는 개발자가 수동으로 수정해야 합니다. 패치 결과의 검증 부재: 기존 AI 패치 도구는 수정 코드를 제안하더라도 보안 문제가 실제로 해결됐는지, 기존 기능이 유지되는지까지 증명하지 못합니다.

아이디어	핵심 컨셉: AI 가 패치를 단순히 ‘작성’하는 데 그치지 않고, 패치가 문제를 해결했음을 TDD 기반 테스트 (FAIL → PASS)로 직접 증명한 뒤 GitHub PR 을 생성하는 자율 검증 파이프라인을 구축합니다. [MVP 스코프 집중] : 백엔드/프론트 범위를 과도하게 넓히지 않고 알려진 취약 Dependency 탐지와 대표적 코드 보안 패턴 검사, 두 가지 핵심 파이프라인의 완성도에 집중합니다. [Multi-Agent 설계] : - Agent 1 : Trivy/OSV-Scanner(SCA) 및 Semgrep(SAST) 도구를 MCP Tool 로 래핑하여 실행. 출력된 파싱 결과(JSON)를 에이전트가 직접 수집·해석하여 오케스트레이션하고, 실제 패치가 필요한 취약 코드 및 의존성 후보 확정. - Agent 2 : NVD/CVE 및 GitHub Security Advisory MCP 를 연동하여 발견된 취약점의 실제 심각도(Severity) 및 하위 호환 가능 권장 버전 정밀 검증. - Agent 3 : 기존 서비스 비즈니스 로직을 보존하며, Semgrep 이 지적한 취약 코드 패턴을 안전한 코드 구문으로 리팩토링 및 라이브러리 업데이트. - Agent 4 : 보안 패치 적용 후 → GitHub PR 생성 [MCP 활용] : AI 의 단순 답변 생성을 넘어 실제 GitHub Repository 와 외부 보안 DB API 를 읽고, TDD 검증을 거친 최종 패치 결과를 다시 GitHub PR 로 안전하게 연동하기 위해 MCP 구조를 적용합니다. [활용 기술/프로그램] : Claude Code, MCP, Multi-Agent, TDD, Code Review, GitHub Repository/PR, NVD·CVE, GitHub Advisory.
기대효과	[사회적 효과] 보안 코딩 문화의 자연스러운 정착: AI 기반 개발 과정에서 놓치기 쉬운 보안 검토를 자동화하여, 보안 전문 지식이 부족한 개발자도 개발 초기 단계부터 자연스럽게 안전한 코드를 작성하는 문화를 형성합니다. 오픈소스 및 AI 생태계 안전성 기여: AI 생성을 통해 무분별하게 양산될 수 있는 보안 취약점의 실서비스 유출을 차단하여, 안전한 AI 소프트웨어 공급망 환경 구축에 기여합니다. [경제적 효과] 보안 점검 및 수정 공수 절감: 단순 경고 확인과 수동 패치에 소요되는 개발자 공수를 자동화하여 개발 시간을 단축하고, 배포 후 뒤늦게 취약점이 발견되어 발생하는 막대한 사후 재작업 비용을 줄입니다. 프로덕션 장애 및 리팩토링 손실 최소화: TDD 기반 검증을 거쳐 패치를 제안하므로, 보안 수정 과정에서 발생하는 기존 기능의 하위 호환성 파괴 위험을 사전 차단하여 서비스 장애 손실을 예방합니다. [일상생활·개발환경 개선 및 변화] 경고 피로감 없는 실질적 패치 경험: 단순히 취약점 목록만 나열되어 개발자에게 부담을 주던 기존 도구와 달리, [재현 테스트 → 패치 → 검증 성공] 내역이 명확히 담긴 Git PR 을 받아 수정 근거를 즉시 확인하고 반영할 수 있습니다. 입문자의 안전한 Vibe Coding 진입: AI 코딩 경험이나 보안 지식이 많지 않은 개발자도 "탐지 → 실제 취약 여부 확인 → 패치 → TDD 검증" 파이프라인을 그대로 활용하여 안심하고 결과물을 배포할 수 있습니다. 속도와 보안의 일체화: 개발 속도 향상과 보안 검증을 서로 충돌하는 트레이드오프 관계가 아닌, 개발 흐름 안에서 자연스럽게 돌아가는 하나의 자동화된 프로세스로 연동합니다.